Unidad 1.
Fundamentos del aprendizaje automático avanzado

Unidad 1
Fundamentos del aprendizaje automático avanzado

Contenido	Página
1. Bienvenida y presentación del tema……	3
2. Introducción al tema de la asignatura……	4
3. Desarrollo del contenido……	5
3.1 Comprender los fundamentos conceptuales y matemáticos que explican el funcionamiento de los principales paradigmas del aprendizaje automático (supervisado, no supervisado y por refuerzo)……	5
3.2 Identificar los factores que influyen en la complejidad y rendimiento de los modelos, incluyendo el sesgo, la varianza y el sobreajuste……	17
3.3 Aplicar procesos de preparación, limpieza y transformación de datos para asegurar la calidad del entrenamiento y la validez de los resultados……	20
4. Referencias bibliográficas……	23
5. Bibliografía……	23

1. Bienvenida y presentación del tema

¡Hola, bienvenidos y bienvenidas!

Nos complace darles la bienvenida a la Unidad 1 de la asignatura **Aprendizaje Automático Avanzado**, en la cual iniciaremos el estudio de los fundamentos que sustentan el desarrollo, análisis y evaluación de modelos inteligentes utilizados en contextos reales y complejos.

Comenzamos con un tema esencial que constituye la base de todo el curso: los fundamentos teóricos, matemáticos y metodológicos del aprendizaje automático avanzado. Esta unidad sienta las bases conceptuales necesarias para comprender cómo y por qué funcionan los distintos paradigmas del aprendizaje automático, así como los factores que influyen directamente en su desempeño.

¿Por qué empezamos aquí?

Es común asociar el aprendizaje automático con modelos sofisticados, arquitecturas profundas o algoritmos de alto impacto. Sin embargo, detrás de cada modelo avanzado existe una comprensión sólida de principios fundamentales como el aprendizaje supervisado, no supervisado y por refuerzo, así como conceptos clave relacionados con la complejidad del modelo, el sesgo, la varianza y el sobreajuste. Sin este entendimiento, la aplicación de técnicas avanzadas carece de sustento y rigor.

De manera análoga, un estudiante de posgrado puede disponer de herramientas computacionales de última generación, pero si no comprende los principios que gobiernan el sistema que analiza, los resultados obtenidos serán limitados o incluso incorrectos. En aprendizaje automático, la correcta preparación, limpieza y transformación de los datos, junto con una comprensión teórica sólida, son elementos indispensables para garantizar modelos confiables y válidos.

La competencia general de esta unidad consiste en analizar los fundamentos teóricos, matemáticos y metodológicos del aprendizaje automático avanzado, con el propósito de evaluar el comportamiento, la complejidad y el rendimiento de modelos supervisados, no supervisados y por refuerzo, aplicando técnicas adecuadas de preparación y transformación de datos que aseguren la calidad del entrenamiento y la validez de los resultados.

En esta Unidad 1. Fundamentos del aprendizaje automático avanzado, nos enfocaremos en el estudio de los siguientes temas:

- Los paradigmas fundamentales del aprendizaje automático: supervisado, no supervisado y por refuerzo.
- Los factores que influyen en la complejidad y el rendimiento de los modelos, como el sesgo, la varianza y el sobreajuste.

- Los procesos de preparación, limpieza y transformación de datos como base para un entrenamiento adecuado y resultados confiables.

Al finalizar esta unidad, se espera que desarrollen las siguientes competencias específicas:

- Comprender los fundamentos conceptuales y matemáticos que explican el funcionamiento de los principales paradigmas del aprendizaje automático.
- Identificar los factores que influyen en la complejidad y el rendimiento de los modelos, analizando fenómenos como el sesgo, la varianza y el sobreajuste.
- Aplicar procesos de preparación, limpieza y transformación de datos que garanticen la calidad del entrenamiento y la validez de los resultados obtenidos.

Recuerda que puedes resolver tus dudas y dar seguimiento a tu aprendizaje a través de los canales de comunicación proporcionados por tu profesor.

2. Introducción al tema de la asignatura

Estimadas y estimados estudiantes:

Bienvenidos al inicio formal del estudio del Aprendizaje Automático Avanzado, una de las áreas centrales de la Inteligencia Artificial moderna. Esta primera unidad, Fundamentos del aprendizaje automático avanzado, ha sido concebida como el punto de articulación entre los principios teóricos y matemáticos que sustentan los modelos inteligentes y su aplicación rigurosa en contextos reales y de alta complejidad.

En este tema abordaremos los fundamentos conceptuales que explican el funcionamiento de los principales paradigmas del aprendizaje automático: el aprendizaje supervisado, no supervisado y por refuerzo. Estos enfoques constituyen la base sobre la cual se construyen modelos capaces de aprender a partir de los datos, generalizar patrones y tomar decisiones en escenarios dinámicos e inciertos.

Asimismo, se analizarán los factores que influyen directamente en el comportamiento y el rendimiento de los modelos, tales como el sesgo, la varianza y el sobreajuste, elementos clave para comprender las limitaciones, riesgos y alcances de los algoritmos avanzados. Este análisis permitirá desarrollar una visión crítica sobre el diseño y la evaluación de modelos, más allá de su simple implementación.

De manera complementaria, se estudiarán los procesos de preparación, limpieza y transformación de datos, reconociendo que la calidad del entrenamiento y la validez de los resultados dependen, en gran medida, de un tratamiento adecuado de la información de entrada. Estos procesos constituyen una etapa indispensable en cualquier flujo de trabajo de aprendizaje automático a nivel profesional y de investigación.

A lo largo de esta unidad, se espera que el estudiante desarrolle la capacidad de:

- Comprender los fundamentos teóricos y matemáticos que rigen los distintos paradigmas del aprendizaje automático.
- Analizar la complejidad y el rendimiento de los modelos a partir de conceptos como sesgo, varianza y sobreajuste.
- Aplicar técnicas de preparación y transformación de datos como base para el desarrollo de modelos confiables y reproducibles.

Al concluir esta introducción, el estudiante contará con una base sólida para abordar, en las unidades posteriores, técnicas más avanzadas como el aprendizaje por refuerzo profundo, los modelos generativos y el despliegue de sistemas inteligentes en entornos reales, manteniendo siempre una perspectiva crítica, ética y técnicamente fundamentada.

3. Desarrollo

3.1 Comprender los fundamentos conceptuales y matemáticos que explican el funcionamiento de los principales paradigmas del aprendizaje automático (supervisado, no supervisado y por refuerzo).

3.1.1 Aprendizaje automático como modelado y optimización

El aprendizaje automático puede entenderse como el proceso mediante el cual un sistema construye modelos matemáticos capaces de aprender patrones a partir de datos y generalizar dicho aprendizaje a situaciones no observadas. En lugar de programar reglas explícitas, el aprendizaje automático busca inferirlas a partir de ejemplos, ajustando un modelo para mejorar su desempeño en una tarea (Mitchell, 1997).

En un nivel de maestría en Inteligencia Artificial, este proceso se interpreta como un problema de modelado y optimización: se define cómo representar los datos, qué objetivo se desea alcanzar y cómo se ajustarán los parámetros del modelo para lograrlo. De manera general, esto implica:

- Representación: transformar el fenómeno observado en variables numéricas procesables.
- Función objetivo: definir una medida de desempeño (pérdida, utilidad o recompensa).
- Optimización: ajustar parámetros del modelo para minimizar una pérdida o maximizar una utilidad (Goodfellow et al., 2016).

Bajo este marco común se articulan los tres paradigmas fundamentales del aprendizaje automático: supervisado, no supervisado y por refuerzo, los cuales comparten herramientas matemáticas (álgebra lineal, probabilidad, estadística, optimización), pero difieren en la forma en que se formula el problema y en el tipo de retroalimentación disponible.

3.1.2 Aprendizaje supervisado: de los datos etiquetados a una función de decisión

En el aprendizaje supervisado, el modelo se entrena con datos etiquetados. Esto significa que cada instancia de entrada tiene asociada una salida conocida. El objetivo es aprender una función que aproxime la relación entre entradas y salidas, y que, además, generalice a datos no vistos (Hastie et al., 2009).

a) Representación matemática del texto

En el caso de estudio, el contenido textual de cada página web se transforma en un vector numérico:

$$x_i = (f_1, f_2, \ldots, f_n)$$

donde cada fj representa la frecuencia de un término relevante del dominio. Esta idea es central: el modelo no aprende "del texto", sino de una representación matemática del lenguaje, construida a partir de características cuantificables.

b) Función objetivo y aprendizaje del modelo

La variable Correcta define la etiqueta y que el modelo debe predecir (por ejemplo, pertenencia o no al contexto de cibersuicidio). En consecuencia, el aprendizaje supervisado busca una función parametrizada:

$$f_\theta(x) \to y$$

donde θ representa los parámetros del modelo. El proceso de entrenamiento ajusta θ minimizando una función de pérdida, con el propósito de reducir los errores de clasificación y mejorar la generalización.

Caso de estudio como ejemplo

Para ilustrar de manera consistente los fundamentos teóricos y matemáticos de esta unidad, se utilizará un caso de estudio aplicado, sobre la Detección de cibersuicidio en páginas web.

En este caso, se cuenta con un conjunto de datos generado mediante un proceso de extracción, transformación y carga de información proveniente de la Web. Cada registro representa una página web, y las variables predictoras corresponden a frecuencias de palabras extraídas del contenido textual. La variable objetivo, denominada Correcta, indica si la página sí pertenece o no pertenece al contexto de cibersuicidio.

Este caso es particularmente útil para la unidad 1 porque conecta de forma natural:

- la representación matemática del lenguaje,
- la formulación del aprendizaje como optimización,
- y la importancia de la validez y la generalización.

A continuación, se presenta la estructura del dataset, así como el código correspondiente en lenguaje Python, con el propósito de que los estudiantes puedan replicar y analizar los experimentos realizados. No obstante, es importante enfatizar que el objetivo central del apartado 3.1 no es la implementación técnica en sí misma, sino la comprensión de la formulación conceptual y matemática que sustenta cada uno de los paradigmas del aprendizaje automático abordados.

El dataset está conformado por registros obtenidos a partir de páginas web, donde cada fila representa una página individual y cada columna corresponde a la frecuencia de términos relevantes extraídos de su contenido textual. La variable objetivo indica si la página pertenece o no al contexto de cibersuicidio, lo que permite formular el problema como una tarea de clasificación supervisada. El conjunto de datos, mostrado en la Figura 1.1, está integrado por 1 157 sitios web y 97 características, las cuales representan indicadores léxicos asociados al dominio del cibersuicidio. Dichas características se presentan agrupadas y clasificadas en la Figura 1.2, con el propósito de facilitar su interpretación y análisis.

El archivo del dataset se encuentra disponible en la sección de recursos con el nombre "dataset_cibersuicidio.csv".

1	suicidio	matarse	muerte	volunta...	quitarse	vida	provocar	intencio...	fatalista	ataques	suicidas	kamika...	egoi
2	266	41	30	1	5	57	5	0	7	1	588	0	1
3	269	30	37	3	3	70	3	1	5	6	506	3	2
4	269	30	37	3	3	70	3	1	5	6	506	3	2
5	269	30	37	3	3	70	3	1	5	6	506	3	2
6	269	30	37	3	3	70	3	1	5	6	506	3	2
7	269	30	37	3	3	70	3	1	5	6	506	3	2
8	234	62	85	26	23	111	3	0	2	1	359	0	0
9	231	14	32	12	5	38	11	3	2	0	345	1	0
10	220	11	12	2	6	25	2	0	2	0	286	0	0
11	184	21	111	24	7	73	0	0	2	1	284	0	0
12	4	87	10	0	2	50	7	0	0	0	0	0	0
13	128	9	33	1	9	56	3	0	0	0	211	0	4
14	77	11	38	0	33	59	13	0	0	1	126	0	0
15	34	10	8	0	7	64	3	0	2	3	87	0	3
16	40	0	9	0	0	41	0	0	1	0	223	0	0
17	75	29	8	5	0	44	1	2	4	0	209	0	0

PREVIEW

Figura 1.1 Dataset de cibersuicidio

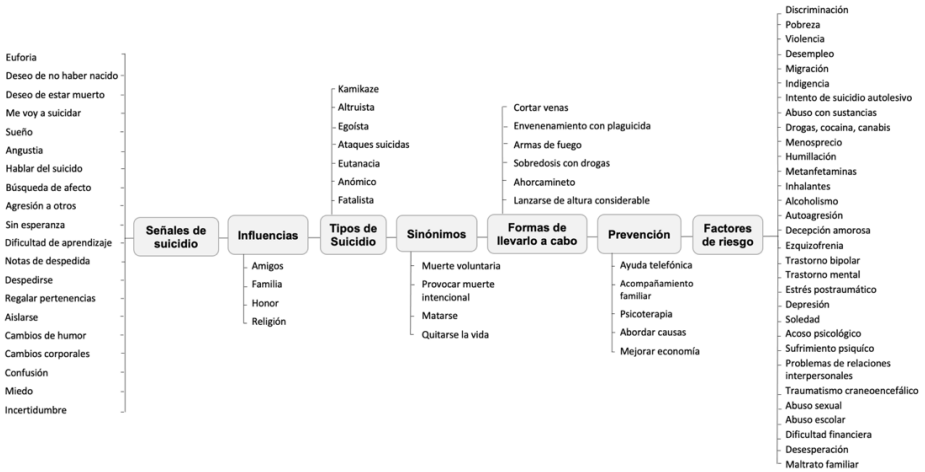


Figura 1.2 Clasificación de palabras clave o características del cibersuicidio (Castillo-Zúñiga et al. 2022)

El ejemplo de aprendizaje supervisado que se presenta a continuación consiste en una red neuronal implementada en Python, diseñada para ilustrar el proceso de entrenamiento, evaluación y generalización de un modelo supervisado. Dicho código puede ejecutarse en entornos como Visual Studio Code, Jupyter Notebook u otra herramienta de desarrollo equivalente, de acuerdo con la preferencia del estudiante.

Programa con red neuronal

```
# Librerías
# ********************************************************

# Librería para el tiempo
import time

# Librería para algoritmos de machine learning
import sklearn
# Importamos el train_test_split para dividir los datos de entrenamiento y prueba
from sklearn.model_selection import train_test_split
# Especificaciones para usar la Red Neuronal Perceptrón Multicapa
from sklearn.neural_network import MLPClassifier
# Librería para construir la matriz de confusión
from sklearn.metrics import confusion_matrix

# Librería para ignorar los warnings
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Librería para cargar el dataset o dataframe
import pandas as pd
import seaborn as sns

# Librería para usar graficación de la matríz de confusión
import matplotlib.pyplot as plt

# Especificación del algoritmo a utilizar
print("\n")
print ("-----------------------------------------------------------------------------------------")
print ("                    MODELO DE CLASIFICACIÓN CON UNA RED NEURONAL")
print ("-----------------------------------------------------------------------------------------")
print("\n")
print ("                     DETECCIÓN DE VOCABULARIO DE CIBERSUICIDIO")

# Proceso del algoritmo de Red Neuronal
# ********************************************************

# Inicia del contador de tiempo
inicio_tiempo=time.time()

# Carga el dataset
dataset = 'dataset_suicidio.csv'
df = pd.read_csv(dataset)

print("\n")
print ("Dimensión del Dataset: ")
print ("Registros: ", df.shape[0])
print ("Columnas:  ", df.shape[1])
print("\n")
print ("Variables predictoras (Valores)")
arreglox = df[df.columns[:-1]].values # Específica tomar variables predictoras
print (arreglox)
print("\n")
print ("Variable a predecir (Valores)")
arregloy= df[df.columns[-1]].values # Específica tomar variable a predecir (última posición)
print (arregloy)
```

```
#Muestra las características utilizadas en el análisis
print("\n")
print("Características utilizadas en el análisis")
print (df.keys())

# Especificación del modelo de datos para entrenamiento
X_train, X_test, y_train, y_test = train_test_split(arreglox , arregloy,test_size = 0.3)
print("\n")
print ("Entrenamiento: “, len(X_train), "registros (70%)")
print ("Pruebas:       ",len(X_test), "registros (30%)")

# Variable red carga el modelo de Red Neuronal, parámetros (iteraciones y capas ocultas en la red)
# El modelo se ajusta hasta que se llegue a una solución óptima
red=MLPClassifier(max_iter=11000, hidden_layer_sizes=(4))

# Establece en el modelo número de registros para entrenamiento y para pruebas
red.fit(X_train,y_train)

# Verifica el porcentaje de aprendizaje del algoritmo
print ("Aprendizaje del algoritmo en el entrenamiento: {0:.2f}"
      .format (red.score(X_test,y_test)),"%")
print("\n")

# Especificación del modelo de datos para clasificar (pruebas)
model = MLPClassifier()

# Ajusta el modelo a los datos de entrenamiento
print ("Ajusta el modelo de datos al proceso de entrenamiento")
print (model.fit(X_train, y_train))

#Matríz de confusión de entrenamiento
y_predicted = model.predict(X_test)
cm = confusion_matrix(y_test, y_predicted)

#Matríz de confusión de pruebas
y_pred = red.fit(X_train, y_train).predict(X_test)
cnf_matrix = confusion_matrix(y_test, y_pred)

#Finaliza el tiempo del proceso
tiempo=(time.time()-inicio_tiempo)

# Matriz de confusión de pruebas
print("\n")
print ("MATRIZ DE CONFUSIÓN")
sns.heatmap(cnf_matrix.T, square=True, annot=True, fmt='d', cbar=True)
plt.xlabel('Clase verdadera')
plt.ylabel('Clase predecida')
plt.title('Matriz de Confusión')
plt.show()
print ("Descripción")
print ("Verdaderos Negativos: ",cnf_matrix[0,0])
print ("Verdaderos Positivos: ",cnf_matrix[1,1])
print ("Falsos Positivos:     ",cnf_matrix[1,0])
print ("Falsos Negativos:     ",cnf_matrix[0,1])

# Visualizando los resultados
print("\n\n")
print ("RESULTADOS DE CLASIFICACIÓN")
print ("---------------------------------------------------------------")
porcentaje=(1-(y_test != y_pred).sum()/y_test.shape[0])
print("Porcentaje de precisión:   {0:.1f}"
     .format(porcentaje * 100),"%")
print("Errores de clasificación:  {1} errores, sobre un total de {0} casos"
     .format(y_test.shape[0],(y_test != y_pred).sum()))
print("Tiempo de ejecución es de: {0:.2f}"
     .format(tiempo),"seg")
print ("---------------------------------------------------------------")
```

Resultado del programa

DETECCIÓN DE VOCABULARIO DE CIBERSUICIDIO

Dimensión del Dataset:
Registros: 1157
Columnas: 98

Variables predictoras (Valores)
```
[[266  41  30 ...   2   9   0]
 [269  30  37 ...   1   4   1]
 [269  30  37 ...   1   4   1]
 ...
 [  0   0   0 ...   0   0   0]
 [  0   0   0 ...   0   0   0]
 [  0   0   0 ...   0   0   0]]
```

Variable a predecir (Valores)
['Si' 'Si' 'Si' ... 'No' 'No' 'No']

Características utilizadas en el análisis
```
Index(['suicidio', 'matarse', 'muerte', 'voluntaria', 'quitarse', 'vida', 'provocar', 'intencional', 'fatalista', 'ataques', 'suicidas',
       'kamikaze', 'egoista', 'altruista', 'anomico', 'cortar', 'venas', 'ahorcamiento', 'sobredosis', 'envenenamiento', 'plaguicidas', 'armas',
       'fuego', 'saltar', 'altura', 'ayuda', 'telefonica', 'acompanamiento', 'familia', 'psicoterapia', 'abordar', 'causas', 'mejorar', 'economia',
       'amigos', 'religion', 'honor', 'sentido', 'dificultad', 'aprendizaje',
       'regalar', 'pertenencias', 'aislarse', 'despedirse', 'esperanza', 'agresion', 'sueno', 'busqueda', 'afecto', 'angustia', 'notas',
       'nacido', 'euforia', 'cambios', 'humor', 'corporal', 'confusion', 'miedo', 'incertidumbre', 'discriminacion', 'pobreza', 'violencia',
       'desempleo', 'migracion', 'indigente', 'abuso', 'sustancias', 'drogas', 'cocaina', 'metanfetaminas', 'inhalantes', 'alcoholismo',
       'esquizofrenia', 'trastorno', 'bipolar', 'mental', 'estres', 'postraumatico', 'depresion', 'soledad', 'sufrimiento', 'psiquico',
       'desesperacion', 'financiera', 'problemas', 'relaciones', 'interpersonales', 'acoso', 'psicologico', 'intento', 'autoagresion',
       'traumatismo', 'craneoencefalico', 'decepcion', 'amorosa', 'sexual',  'escolar', 'Correcta'],
      dtype='object')
```

Entrenamiento: 809 registros (70%)
Pruebas: 348 registros (30%)
Aprendizaje del algoritmo en el entrenamiento: 0.98 %

Ajusta el modelo de datos al proceso de entrenamiento
```
/Users/ivancastillozuniga/opt/anaconda3/lib/python3.8/site-packages/sklearn/neural_network/_multilayer_perceptron.py:614:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
MLPClassifier()
```

MATRIZ DE CONFUSIÓN



Descripción
Verdaderos Negativos: 39
Verdaderos Positivos: 302
Falsos Positivos: 3
Falsos Negativos: 4

RESULTADOS DE CLASIFICACIÓN

Porcentaje de precisión: 98.0 %
Errores de clasificación: 7 errores, sobre un total de 348 casos
Tiempo de ejecución es de: 3.37 seg

Este caso de estudio permite observar con claridad que un buen desempeño no se reduce a "acertar" en entrenamiento: se requiere evaluar sobre datos no vistos y considerar métricas que reflejen el significado de los errores en contextos sensibles.

Desde el punto de vista conceptual, este enfoque permite analizar cómo se distribuyen las páginas web en el espacio de características definido por los vectores, identificar agrupamientos naturales o construir representaciones más compactas que reduzcan la complejidad del problema. Matemáticamente, estas técnicas se apoyan en criterios como la distancia, la similitud, la varianza o el error de reconstrucción.

En el contexto del análisis de información web, el aprendizaje no supervisado puede utilizarse como una etapa previa para explorar patrones emergentes en grandes volúmenes de páginas, apoyar la comprensión del dominio o preparar representaciones que posteriormente serán utilizadas en modelos supervisados o de refuerzo.

3.1.3 Aprendizaje no supervisado: estructura latente sin etiquetas

En el aprendizaje no supervisado, no se dispone de etiquetas. El objetivo no es aproximar una función de predicción, sino descubrir estructura en los datos: agrupamientos, patrones, regularidades o representaciones compactas (Murphy, 2012).

Desde el punto de vista matemático, este paradigma parte de las mismas representaciones, pero cambia la naturaleza de la función objetivo: en lugar de minimizar error respecto a una etiqueta, se optimizan criterios como distancia, similitud, varianza explicada o reconstrucción.

Aplicado al caso de estudio del cibersuicidio del objetivo anterior, el aprendizaje no supervisado puede utilizarse para explorar cómo se organizan las páginas en el espacio de características, identificar grupos de páginas con patrones léxicos similares, o construir representaciones más compactas que faciliten el análisis posterior o el modelado supervisado.

En este ejemplo, se emplea PCA para construir una representación de baja dimensión a partir de las frecuencias de términos de cada página web, y posteriormente se aplica K-means para identificar agrupamientos naturales. La etiqueta Correcta no se utiliza para entrenar, ya que el objetivo del aprendizaje no supervisado es descubrir estructura latente; sin embargo, puede emplearse al final únicamente para interpretar qué tan relacionados están los clústeres con el contexto de cibersuicidio.

Programa de aprendizaje no supervisado con PCA y KMeans

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# 1) Cargar dataset (ajusta la ruta si es necesario)
df = pd.read_csv("dataset_cibersuicidio.csv")
```

```
# 2) Separar variables predictoras (frecuencias) y NO usar la etiqueta
#    En no supervisado, 'Correcta' solo se puede usar después para interpretar, no para entrenar.
X = df.drop(columns=["Correcta"], errors="ignore")

# 3) Escalamiento (importante para PCA y K-means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4) PCA a 2 componentes para visualizar estructura latente
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print("Varianza explicada por PC1 y PC2:", pca.explained_variance_ratio_)
print("Varianza acumulada (2 componentes):", pca.explained_variance_ratio_.sum())

# 5) K-means sobre el espacio PCA (también podrías usar X_scaled, pero PCA ayuda a visualizar)
k = 3  # puedes cambiar a 2, 3, 4 para comparar
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_pca)

# 6) Resultados básicos: tamaño de clusters
cluster_counts = pd.Series(clusters).value_counts().sort_index()
print("\nTamaño por cluster:")
print(cluster_counts)

# 7) Visualización simple en 2D
plt.figure()
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, s=15)
plt.title(f"PCA (2D) + K-means (k={k}) - Páginas web")
plt.xlabel("Componente principal 1")
plt.ylabel("Componente principal 2")
plt.show()

# 8) Interpretación: ¿qué términos caracterizan a cada cluster?
#    Calculamos la media de frecuencias por cluster usando las variables originales (X).
X_with_cluster = X.copy()
X_with_cluster["cluster"] = clusters

cluster_means = X_with_cluster.groupby("cluster").mean(numeric_only=True)

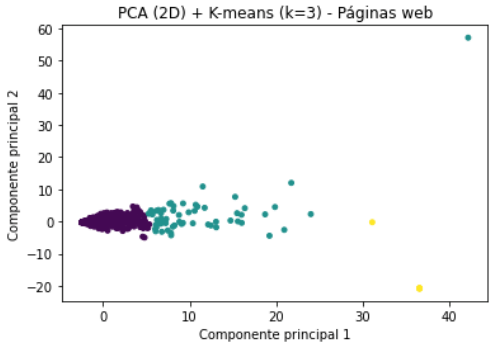
# Mostrar las 10 características más altas por cluster (en promedio)
top_n = 10
for c in range(k):
    top_features = cluster_means.loc[c].sort_values(ascending=False).head(top_n)
    print(f"\nTop {top_n} términos por frecuencia promedio en cluster {c}:")
    print(top_features)

# 9) (Opcional, solo para análisis posterior) Comparar con la etiqueta para interpretar
#    Esto NO es entrenamiento, solo es interpretación de los grupos encontrados.
if "Correcta" in df.columns:
    df_tmp = df.copy()
    df_tmp["cluster"] = clusters
    crosstab = pd.crosstab(df_tmp["cluster"], df_tmp["Correcta"], normalize="index")
    print("\nDistribución de la etiqueta 'Correcta' por cluster (solo interpretación):")
    print(crosstab)
```

Resultados del programa

```
Varianza explicada por PC1 y PC2: [0.1831769  0.06125512]
Varianza acumulada (2 componentes): 0.24443201335719306

Tamaño por cluster:
0    1091
1      60
2       6
dtype: int64
```




```
Top 10 términos por frecuencia promedio en cluster 0:
suicidas     13.247479
suicidio     10.210816
vida          5.060495
muerte        2.959670
ayuda         2.002750
mental        1.807516
sentido       1.674610
matarse       1.508708
familia       1.467461
problemas     1.327223
Name: 0, dtype: float64

Top 10 términos por frecuencia promedio en cluster 1:
suicidas     113.083333
suicidio      61.016667
vida          31.133333
sentido       20.716667
muerte        16.050000
familia       15.166667
mental        14.266667
problemas     11.016667
ayuda         10.666667
depresion     10.566667
Name: 1, dtype: float64

Top 10 términos por frecuencia promedio en cluster 2:
suicidas      519.666667
suicidio      268.500000
vida           67.833333
ayuda          65.333333
mental         41.000000
muerte         35.833333
matarse        31.833333
causas         30.833333
trastorno      29.000000
relaciones     26.166667
Name: 2, dtype: float64

Distribución de la etiqueta 'Correcta' por cluster (solo interpretación):
Correcta        No        Si
cluster
0         0.150321  0.849679
1         0.000000  1.000000
2         0.000000  1.000000
```

Al ejecutar este código de aprendizaje no supervisado, observe que el modelo no utiliza la etiqueta *Correcta* durante el entrenamiento. El objetivo no es clasificar las páginas web, sino explorar cómo se organizan naturalmente en función de la similitud de las frecuencias de términos que las describen.

La reducción de dimensionalidad mediante PCA permite resumir parcialmente la información contenida en las variables originales y facilita la visualización de patrones generales, aun cuando no se conserve toda la variabilidad del conjunto de datos. Posteriormente, K-means identifica agrupamientos naturales que reflejan distintos niveles de intensidad temática en el contenido de las páginas.

Los clústeres obtenidos muestran que algunas páginas presentan un perfil léxico moderado, mientras que otras concentran de forma más marcada términos asociados al dominio del cibersuicidio. La comparación posterior con la etiqueta *Correcta* se realiza únicamente con fines interpretativos, para analizar la coherencia de los agrupamientos, pero no forma parte del proceso de aprendizaje**.**

Este ejercicio ilustra cómo el aprendizaje no supervisado permite descubrir estructura latente en los datos y apoyar la comprensión del dominio, funcionando como un complemento natural al aprendizaje supervisado abordado en esta unidad.

3.1.4 Aprendizaje por refuerzo: del etiquetado a la toma de decisiones secuenciales

En el aprendizaje por refuerzo, el sistema no aprende a partir de etiquetas directas como en el aprendizaje supervisado, sino mediante la interacción de un agente con un entorno, recibiendo una recompensa que guía la mejora de su comportamiento. El objetivo es aprender una política $\pi\ (a \mid s)$ que maximice el retorno esperado (Sutton & Barto, 2018).

Para ilustrar este paradigma se utilizará el entorno FrozenLake, un escenario clásico donde el agente debe aprender a desplazarse desde un estado inicial hasta un estado meta evitando caer en estados de fallo. Este entorno permite definir con claridad los elementos fundamentales del aprendizaje por refuerzo:

- **Estado s**: la posición del agente en el lago (codificada como un entero).
- **Acciones a**: desplazamientos posibles $\{0, 1, 2, 3\}$ correspondientes a izquierda, abajo, derecha y arriba.
- **Recompensa r**: típicamente r=1 si se alcanza la meta y r=0 en otro caso; el episodio termina si se llega a la meta o si se cae en un agujero.
- **Recompensa r**: típicamente r=1 si se alcanza la meta y r=0 en otro caso; el episodio termina si se llega a la meta o si se cae en un agujero.
- **Transición s→s′**: el entorno determina el siguiente estado en función del estado actual y la acción.

Desde el punto de vista matemático, un enfoque fundamental es **Q-Learning**, donde se aprende una función de valor acción Q(s,a) que estima la utilidad esperada de ejecutar la acción a en el estado s. La actualización central se expresa como:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

donde α es la tasa de aprendizaje y γ el factor de descuento.
A continuación, se presenta el código en Python para que el estudiante replique el entrenamiento del agente, conectando la formulación conceptual anterior con una implementación funcional.

Debe instalar las siguientes librerías primero, ya sea utilizando Pip o Pip3

a. Pip install gymnasium
b. Pip install numpy

Programa de aprendizaje por refuerzo con Q-Learning y FrozenLake (Gymnasium)

```
import numpy as np
import gymnasium as gym

# 1) Crear entorno
# is_slippery=True hace que el entorno sea estocástico (más realista).
env = gym.make("FrozenLake-v1", is_slippery=True)

n_states = env.observation_space.n
n_actions = env.action_space.n

# 2) Inicializar Q-table
Q = np.zeros((n_states, n_actions))

# 3) Hiperparámetros
alpha = 0.10      # tasa de aprendizaje
gamma = 0.99      # descuento
epsilon = 1.0     # exploración inicial
epsilon_min = 0.05
epsilon_decay = 0.999

episodes = 15000
max_steps = 200

rng = np.random.default_rng(42)

def epsilon_greedy(state, eps):
    if rng.random() < eps:
        return env.action_space.sample()
    return int(np.argmax(Q[state]))
# 4) Entrenamiento
for ep in range(episodes):
    state, _ = env.reset()
    done = False

    for _ in range(max_steps):
        action = epsilon_greedy(state, epsilon)
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        # Actualización Q-Learning
        best_next = np.max(Q[next_state])
        td_target = reward + gamma * best_next
        td_error = td_target - Q[state, action]
        Q[state, action] += alpha * td_error

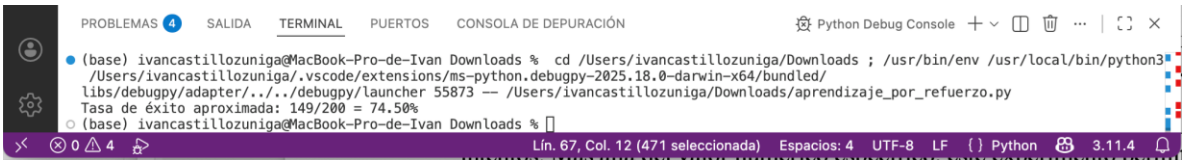
        state = next_state
        if done:
            break

    # Decaimiento de exploración
     epsilon = max(epsilon_min, epsilon * epsilon_decay)
```

```
# 5) Evaluación rápida (sin exploración)
test_episodes = 200
success = 0
for _ in range(test_episodes):
  state, _ = env.reset()
  for _ in range(max_steps):
    action = int(np.argmax(Q[state]))
    state, reward, terminated, truncated, _ = env.step(action)
    if terminated or truncated:
        success += int(reward == 1.0)
      break

print(f"Tasa de éxito aproximada: {success}/{test_episodes} = {success/test_episodes:.2%}")
env.close()
```

Resultados del programa



En la ejecución del algoritmo de Q-Learning sobre el entorno FrozenLake, el agente alcanza una tasa de éxito aproximada del 74.5 **%** en episodios de prueba. Este resultado es consistente con la naturaleza estocástica del entorno (debido a la propiedad is_slippery=True), donde incluso una política adecuada no garantiza el éxito en todos los intentos. Más allá del valor numérico específico, este experimento permite observar cómo la política aprendida se construye a partir de la interacción del agente con el entorno, la definición explícita de estados, acciones y recompensas, y la aplicación de la actualización de Q-Learning, reforzando así los fundamentos conceptuales y matemáticos del aprendizaje por refuerzo presentados en este objetivo.

3.2 Identificar los factores que influyen en la complejidad y rendimiento de los modelos, incluyendo el sesgo, la varianza y el sobreajuste

Después de comprender en el objetivo anterior cómo se formulan los problemas de aprendizaje automático y cuáles son sus fundamentos conceptuales y matemáticos, el siguiente paso natural es analizar por qué un modelo funciona bien o mal cuando se enfrenta a datos reales.

En este objetivo, el énfasis no está en introducir nuevos algoritmos, sino en entender el comportamiento de los modelos. En particular, se busca que el estudiante comprenda que el rendimiento de un modelo depende del equilibrio entre:

- la capacidad del modelo para aprender patrones,
- la cantidad y calidad de los datos disponibles,
- y las decisiones de diseño tomadas durante el entrenamiento.

Este análisis se articula a través de tres conceptos fundamentales del aprendizaje automático: sesgo, varianza y sobreajuste, los cuales explican la mayoría de los problemas observados en modelos aplicados (Hastie et al., 2009).

- **Sesgo**: se refiere al error introducido por asumir un modelo demasiado simple para representar la relación real entre los datos. Modelos con alto sesgo tienden a subajustar (underfitting).
- **Varianza**: se refiere a la sensibilidad del modelo a las variaciones del conjunto de entrenamiento. Modelos con alta varianza se ajustan excesivamente a los datos de entrenamiento y generalizan mal.
- **Sobreajuste**: ocurre cuando un modelo aprende patrones específicos del conjunto de entrenamiento, incluyendo ruido, perdiendo capacidad de generalización.

Estos conceptos están íntimamente relacionados y suelen describirse mediante el compromiso sesgo–varianza, que establece que aumentar la complejidad del modelo reduce el sesgo, pero incrementa la varianza (Géron, 2022).

Para abordar un ejemplo tomaremos como base el dataset de cibersuicidio, abordado en el objetivo 3.1. En este caso se pretende mostrar cómo la complejidad del modelo afecta su desempeño, sin introducir redes profundas ni configuraciones complejas. Para su análisis se empleará un problema de clasificación supervisada sencillo con un modelo de árbol de decisión, variando únicamente su profundidad.

Programa para calcular el sesgo vs varianza con árboles de decisión

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1) Cargar dataset (mismo dominio trabajado previamente)
df = pd.read_csv("dataset_cibersuicidio.csv")

X = df.drop(columns=["Correcta"])
y = df["Correcta"]

# 2) Separar datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42, stratify=y
)

# 3) Modelos con distinta complejidad
profundidades = [1, 3, 10, None]

for d in profundidades:
   modelo = DecisionTreeClassifier(max_depth=d, random_state=42)
   modelo.fit(X_train, y_train)

   acc_train = accuracy_score(y_train, modelo.predict(X_train))
   acc_test = accuracy_score(y_test, modelo.predict(X_test))

   print(f"Profundidad={d}")
   print(f"  Exactitud entrenamiento: {acc_train:.3f}")
   print(f"  Exactitud prueba:       {acc_test:.3f}")
   print("-" * 40)
```

Resultados del programa

```
Profundidad=1
  Exactitud entrenamiento: 0.870
  Exactitud prueba:        0.899
----------------------------------------
Profundidad=3
  Exactitud entrenamiento: 0.959
  Exactitud prueba:        0.943
----------------------------------------
Profundidad=10
  Exactitud entrenamiento: 0.999
  Exactitud prueba:        0.971
----------------------------------------
Profundidad=None
  Exactitud entrenamiento: 1.000
  Exactitud prueba:        0.971
----------------------------------------
```

Al analizar los resultados obtenidos con distintas profundidades del árbol de decisión, observe cómo el aumento de la complejidad del modelo mejora de manera sistemática la exactitud en el conjunto de entrenamiento. Sin embargo, este incremento no siempre se traduce en una mejora equivalente en el conjunto de prueba.

El modelo con profundidad baja presenta un comportamiento asociado al alto sesgo, ya que su simplicidad limita la capacidad de aprendizaje. A profundidades intermedias, el modelo

logra un mejor equilibrio entre sesgo y varianza, reflejado en un buen desempeño tanto en entrenamiento como en prueba. En contraste, al permitir una profundidad elevada o ilimitada, el modelo alcanza una exactitud perfecta en entrenamiento, pero sin mejoras adicionales en prueba, lo que evidencia un sobreajuste**.**

Este ejercicio refuerza la idea de que el rendimiento de un modelo debe evaluarse en función de su capacidad de generalización, y no únicamente por su desempeño sobre los datos con los que fue entrenado.

3.3 Aplicar procesos de preparación, limpieza y transformación de datos para asegurar la calidad del entrenamiento y la validez de los resultados

Después de comprender en el **Objetivo 3.1** cómo se formulan los modelos de aprendizaje automático y en el **Objetivo 3.2** cómo su complejidad afecta su rendimiento y capacidad de generalización, el siguiente paso esencial es garantizar la calidad de los datos sobre los que dichos modelos aprenden.

En aprendizaje automático avanzado, el desempeño de un modelo no depende únicamente del algoritmo utilizado, sino de manera crítica de los procesos de preparación, limpieza y transformación de los datos. Datos incompletos, mal escalados o inconsistentes pueden conducir a conclusiones erróneas, modelos inestables o resultados no reproducibles, aun cuando se utilicen algoritmos sofisticados (Kuhn & Johnson, 2013).

Por ello, este objetivo se centra en aplicar un flujo sistemático de preparación de datos, orientado a asegurar que el entrenamiento del modelo sea válido y que los resultados obtenidos sean interpretables y confiables.

Procesos fundamentales de preparación de datos

De manera general, la preparación de datos en aprendizaje automático incluye las siguientes etapas:

- **Inspección y comprensión del conjunto de datos**: análisis de variables, tipos de datos y distribución.
- **Limpieza:** tratamiento de valores faltantes, inconsistencias y valores atípicos.
- **Transformación:** escalamiento, normalización o codificación de variables.
- **Separación de datos:** división adecuada entre entrenamiento y prueba para evitar fuga de información.

Estos procesos no son opcionales, sino parte integral del diseño del modelo, ya que influyen directamente en la estabilidad del entrenamiento y en la capacidad de generalización del sistema (Hastie et al., 2009).

A continuación se proporciona un ejemplo de como la preparación de los datos impacta el entrenamiento, utilizando el mismo dominio trabajado previamente desde el objetivo 1.1 con el dataset de cibersuicidio (frecuencias de términos en páginas web).

Programa para la preparación y transformación de datos

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# 1) Cargar dataset
df = pd.read_csv("dataset_cibersuicidio.csv")
# 2) Separar variables predictoras y etiqueta
X = df.drop(columns=["Correcta"])
y = df["Correcta"]
```

```
# 3) Inspección rápida
print(X.isnull().sum().head())

# 4) Limpieza: imputación de valores faltantes (si existen)
imputer = SimpleImputer(strategy="mean")
X_imputed = imputer.fit_transform(X)

# 5) Transformación: escalamiento de variables
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

# 6) Separación entrenamiento / prueba
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

print("Datos listos para entrenamiento y evaluación")
```

Resultados del programa

```
suicidio      0
matarse       0
muerte        0
voluntaria    0
quitarse      0
dtype: int64
Datos listos para entrenamiento y evaluación
```

Los resultados indican que las variables analizadas no presentan valores faltantes, lo que sugiere que el conjunto de datos cuenta con una buena calidad inicial. Aun así, el flujo de preparación de datos se aplica de manera sistemática para garantizar que el proceso sea robusto y replicable en otros escenarios donde sí puedan existir inconsistencias.

La inclusión de etapas como la imputación y el escalamiento no solo corrige posibles problemas, sino que asegura condiciones adecuadas para el entrenamiento del modelo, evitando que diferencias en las escalas de las variables influyan indebidamente en los resultados. Este paso confirma que los datos se encuentran listos para el entrenamiento y la evaluación**,** reforzando la importancia de la preparación de datos como base para obtener modelos válidos y confiables.

Síntesis y cierre de la unidad

En esta unidad se abordaron los fundamentos conceptuales, matemáticos y metodológicos que sustentan el aprendizaje automático avanzado, estableciendo una base sólida para el análisis y desarrollo de modelos inteligentes. A través del estudio de los paradigmas supervisado, no supervisado y por refuerzo, se analizó cómo los problemas de aprendizaje pueden formularse como procesos de modelado y optimización, y cómo una misma fuente de datos puede dar lugar a enfoques distintos según el objetivo del sistema.

Asimismo, se identificaron los factores que influyen en la complejidad y el rendimiento de los modelos, destacando el papel del sesgo, la varianza y el sobreajuste en la capacidad de generalización. Mediante ejemplos prácticos, se evidenció que el desempeño de un modelo no depende únicamente del algoritmo seleccionado, sino del equilibrio entre su complejidad, los datos disponibles y las decisiones de diseño adoptadas durante el entrenamiento.

Finalmente, se aplicaron procesos de preparación, limpieza y transformación de datos, reconociendo que la calidad de los datos constituye un elemento central para garantizar la validez y confiabilidad de los resultados. La preparación sistemática de los datos se presentó como una etapa indispensable, previa a cualquier proceso de entrenamiento, y como un componente clave en proyectos reales de aprendizaje automático.

En conjunto, esta unidad proporciona los cimientos teóricos y prácticos necesarios para comprender el comportamiento de los modelos de aprendizaje automático y evaluar críticamente sus resultados. Estos conocimientos servirán como punto de partida para el estudio de técnicas más avanzadas en las unidades posteriores, donde se profundizará en modelos de mayor complejidad, entornos dinámicos y aplicaciones específicas del aprendizaje automático avanzado.

Referencias Bibliográficas

Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Géron, A. (2022). *Hands-on machine learning with scikit-learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling*. Springer.

Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.

Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Vander Maaten, L., Postma, E., & Van den Herik, J. (2009). Dimensionality reduction: A comparative review. *Journal of Machine Learning Research, 10*, 66–71.

Bibliografía

Castillo-Zúñiga, I., et al. (2022). Detección de rasgos en estudiantes con tendencia suicida en Internet aplicando minería Web. Comunicar, pág. 105-117. DOI: https://doi.org/10.3916/C71-2022-08

Flux, J. (2024). Machine Learning Mathematics in Python. Edición Kindle. ASIN: B0D8ZN5L1L

Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and Tensorflow: Concepts, Tools, and Techniques to Build Intelligent Systems. Editorial O'Reilly Media, 3er ed. ISBN:978-1098125974

Huyen, C. (2023). Diseño de Sistemas de Machine Learning. Ed. Marcocombo. ISBN: 978-8426736956.